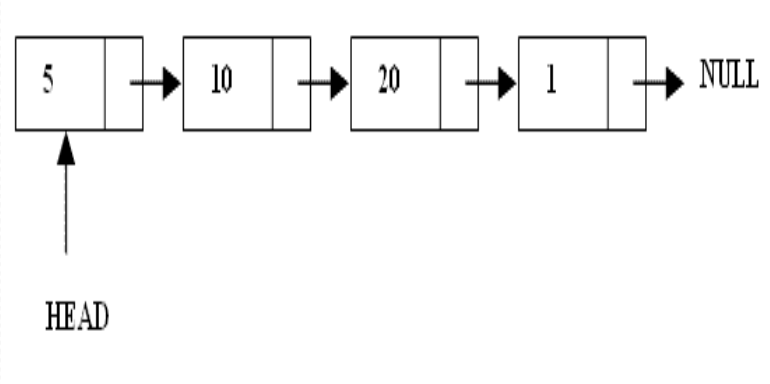
UNIT – III (LINKED LIST)

What are Linked Lists

- A linked list is a linear data structure.
- Nodes make up linked lists.
- Nodes are structures made up of data and a pointer to another node.
- Usually the pointer iscalled next.



Arrays Vs Linked Lists

Arrays	Linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access → Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise may result in much memory waste.	Since memory is allocated dynamically(acc. to our need) there is no waste of memory.
Sequential access is faster [Reason: Elements in contiguous memory locations]	Sequential access is slow [Reason: Elements not in contiguous memory locations]

Types of Linked Lists

- Basically there are 2 typesof linked list
- Singly Linked list
- Doubly linked list

Other types of linked list –

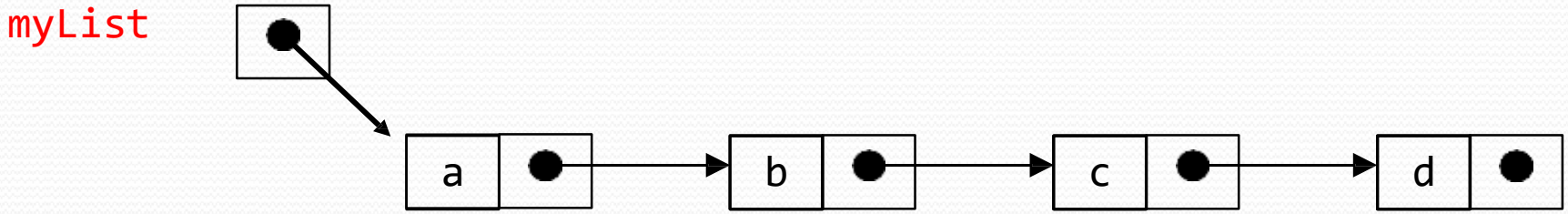
- Circular linked list
- Circular Double linked list
- Header Linked list

Singly Linked List

- Each node has only one link part
- Each link part contains the address of the next node in the list
- Link part of the last node contains NULL value which signifies the end of the node

Schematic representation

- Here is a singly-linked list (SLL):

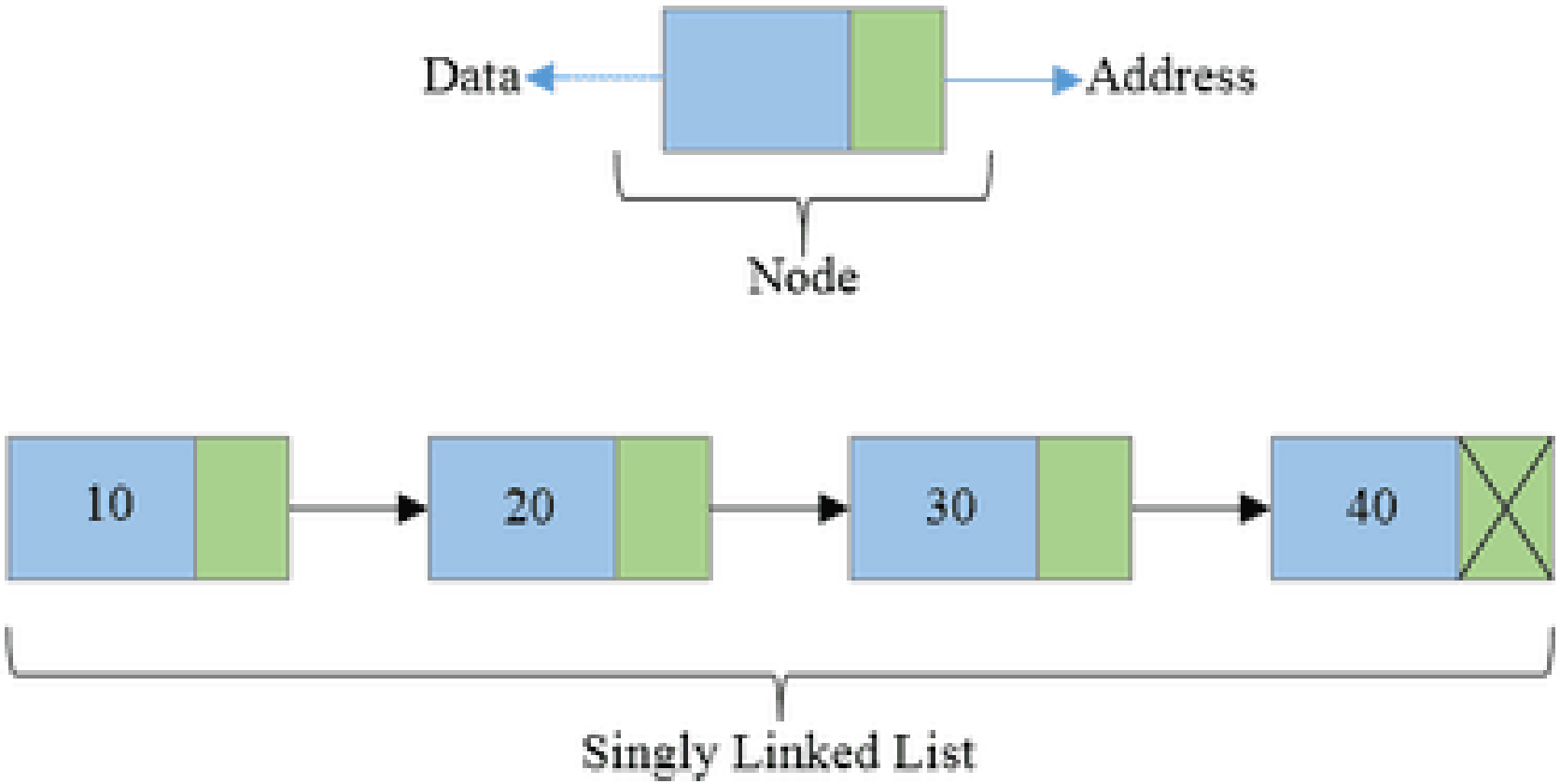


- Each node contains a value (data) and a pointer to the next node in the list

- myList is the header pointer which points at the first node in the list

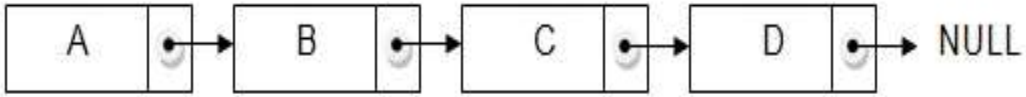
Basic Operations on a list

- Creating a List
- Inserting an element in a list
- Deleting anelement from a list
- Searching a list
- Reversing a list


Data
Address
Node
10
20
30
40
Singly Linked List

Consider Train as a Linked List

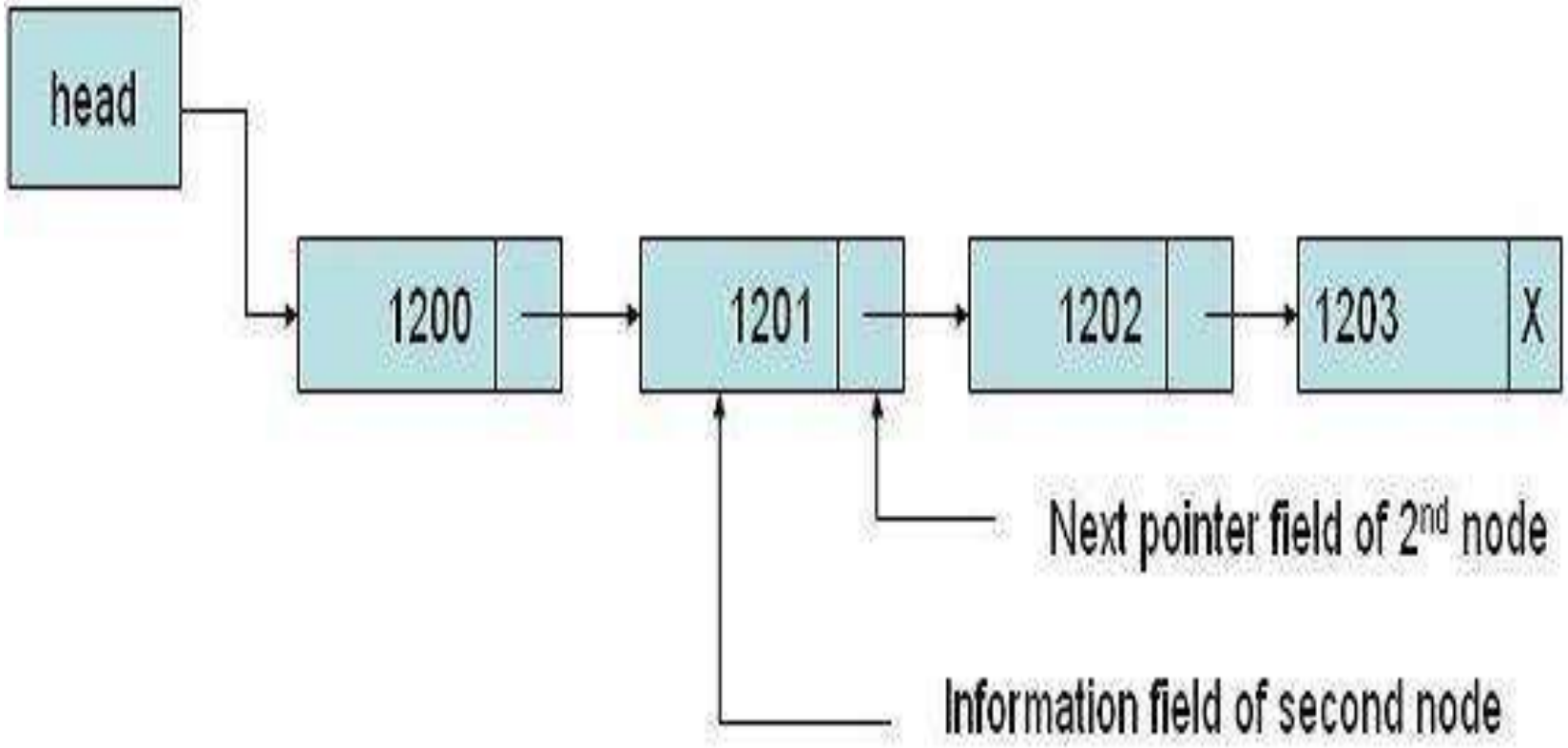
Singly Linked List

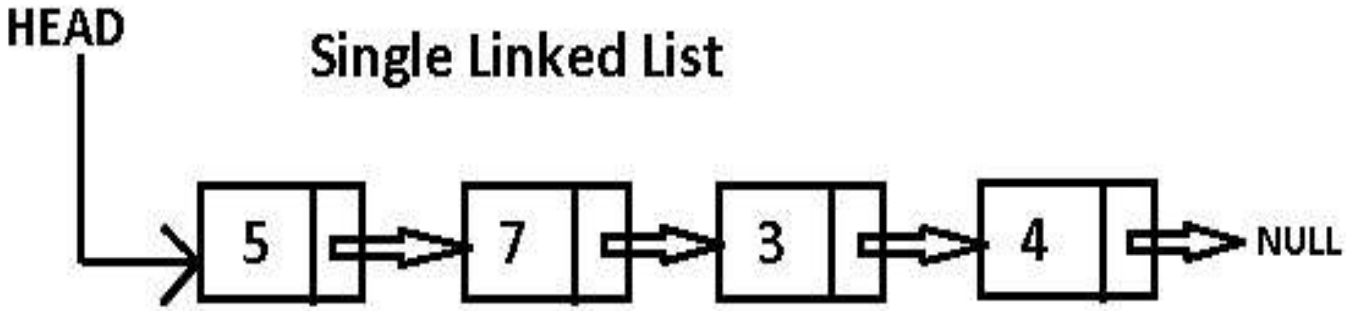


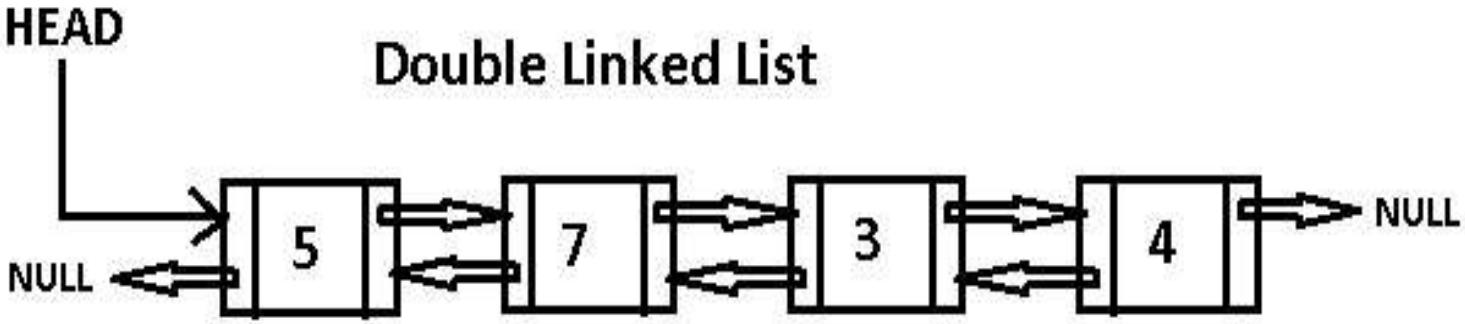
Single Linked List

Linked list

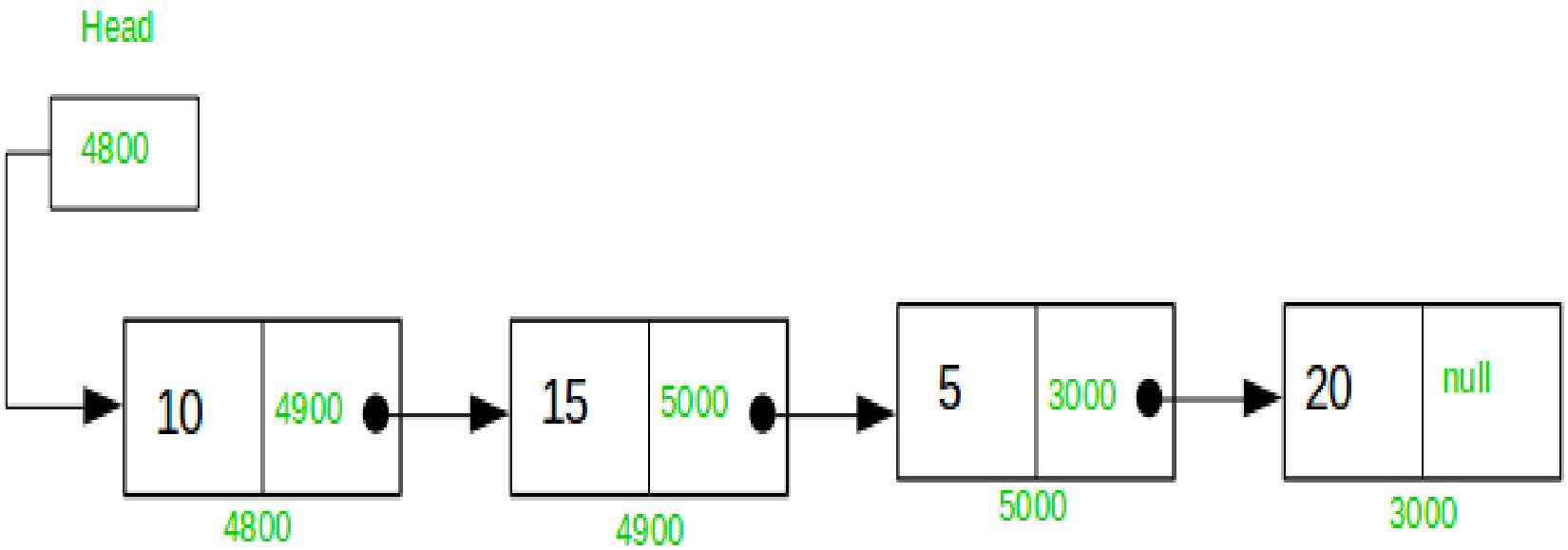
A Proper representation of Linked List

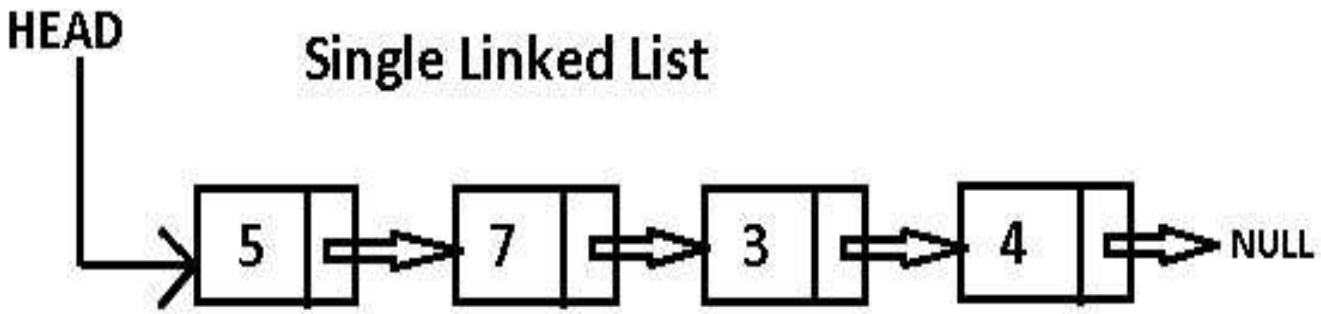


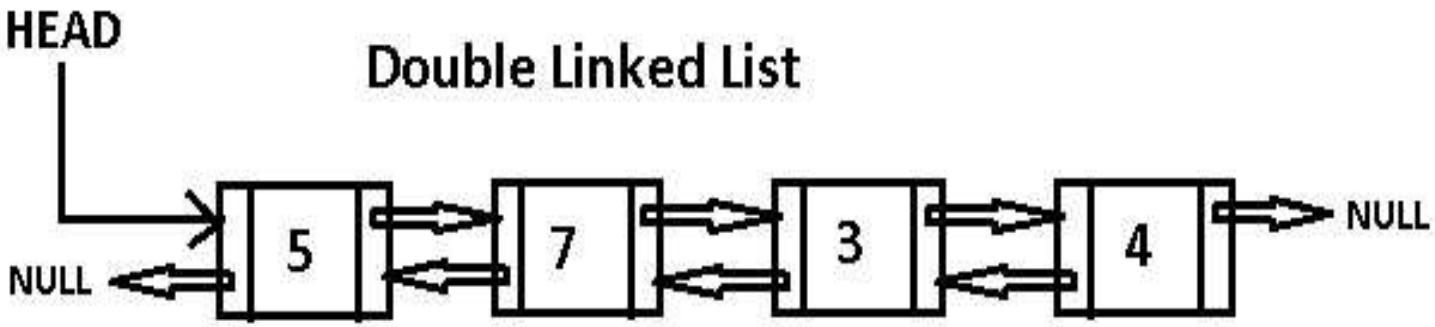

HEAD
Single Linked List
5
7
3
4
NULL

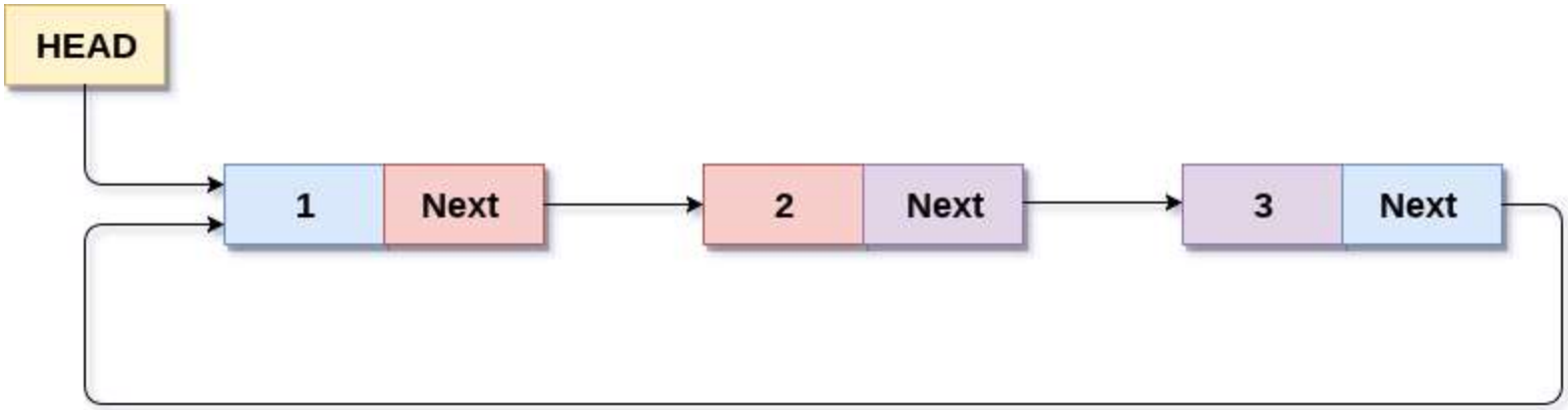

HEAD
Double Linked List
5
7
3
4
NULL
NULL

Singly Linked list




HEAD
Single Linked List
5
7
3
4
NULL


HEAD
Double Linked List
5
7
3
4
NULL
NULL



Circular Singly Linked List

Basic Operations on a list

- Creating a List
- Inserting an element in a list
- Deleting an element from a list
- Searching a list
- Reversing a list

APPLICATIONS OF LINKED LIST

1. Linked list are preferred when we don’t know the volume of data to be stored.
2. Used to implement other Data structures as- Stack, Queue, Graph, Tree, Hash table e.t.c
3. Used in Compiler designing
4. Implementing different file structures
5. Google Search Algorithm
6. Implementing Computer Games
7. Separate Chaining method in Hashing

UNIT – III (LINKED LIST)
Insertion in linked list

Inserting the node in a Single Linked List

There are 3 cases here:-

- Insertion at the beginning
- Insertion at the end
- Insertion after a particular node

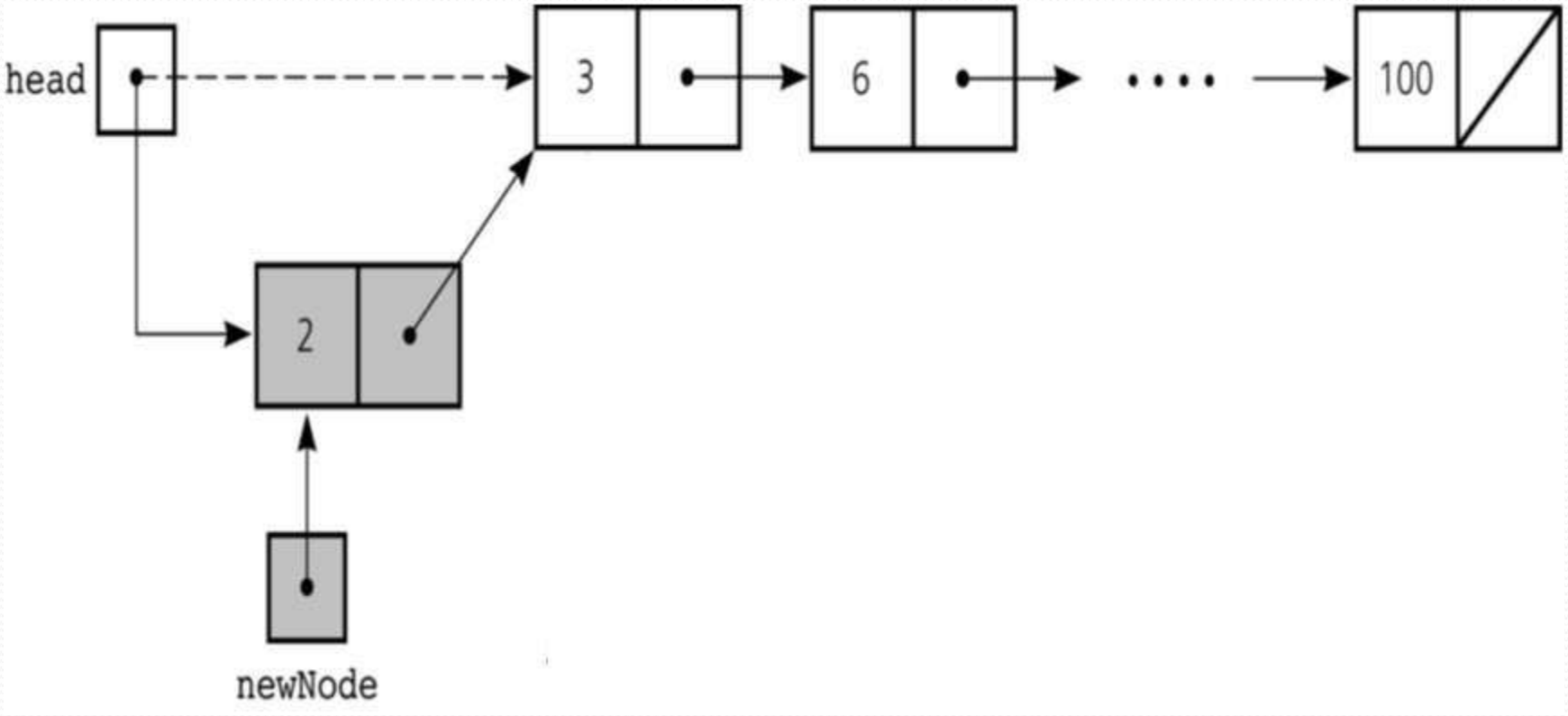
Insertion at the beginning

There are two steps to be followed:-

a) Make the next pointer of the node point towards the first node of the list

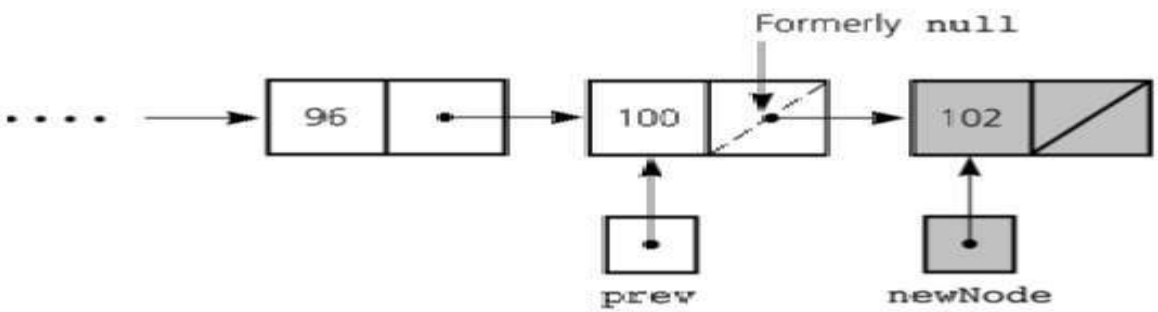
b) Make the start pointer point towards this new node

- If the list is empty simply make the start pointer point towards the new node;


head
3
6
100
2
newNode

Inserting at the end

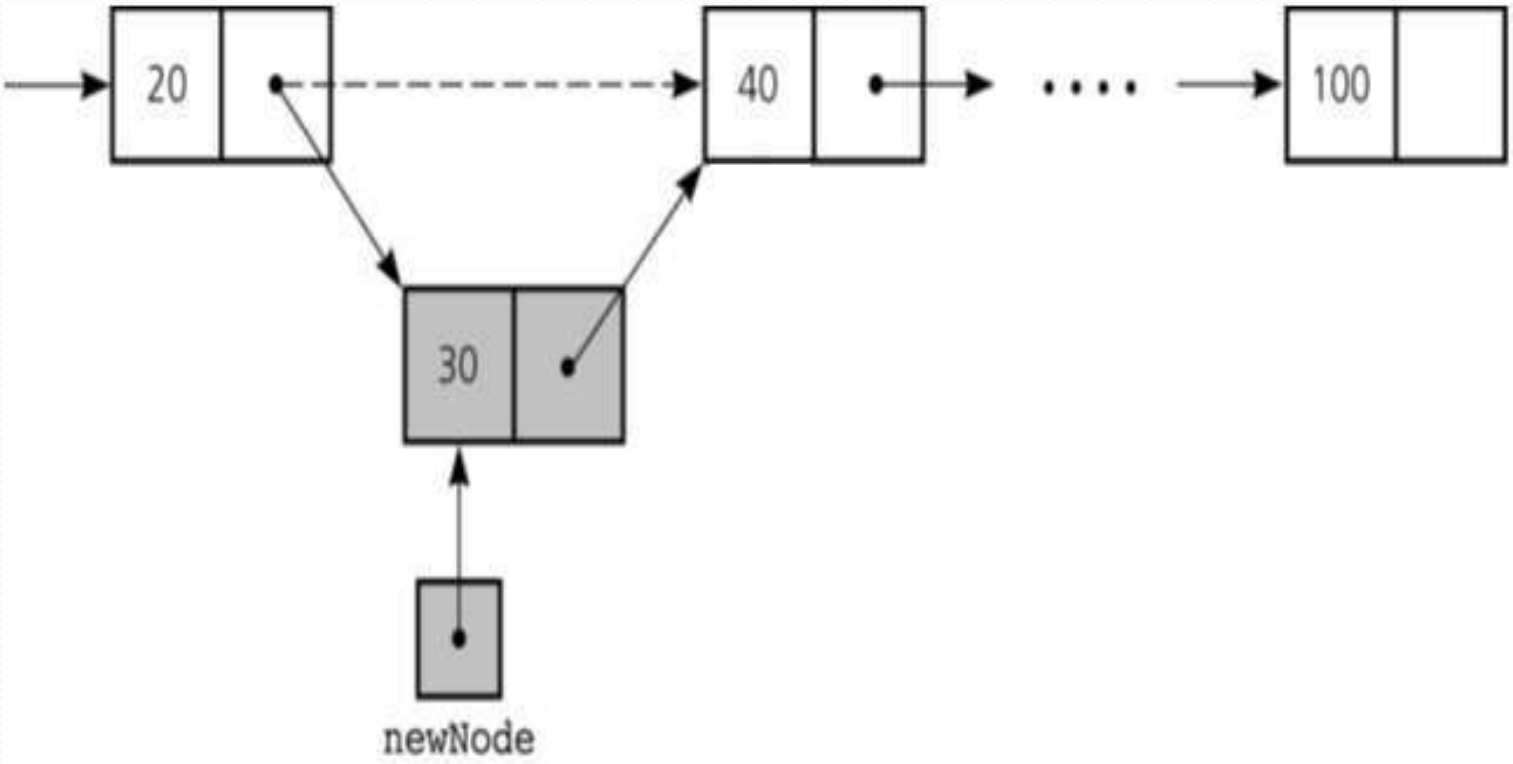
Here we simply need to make the next pointer of the last node point to the new node



Inserting after an node

Here we again need to do 2 steps :-

- Make the next pointer of the node to be inserted point to the next node of the node after which you want to insert the node
- Make the next pointer of the node after which the node is to be inserted, point to the node to be inserted


20
40
100
30
newNode

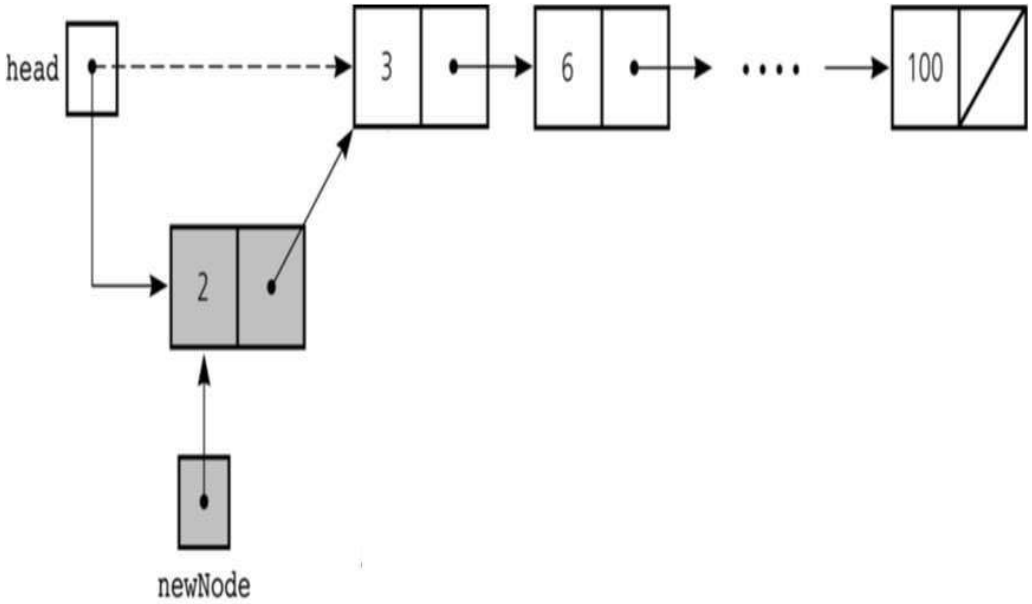
C code for constructing a node

```
new = (struct node*) malloc(size of (struct node))
```

Algorithm for insertion at beg. 1

- Insertion_Beg (temp, data , add, head)

1. temp=(struct node *) malloc(sizeof(struct node))
2. temp->data=data
3. temp->link=NULL
4. temp->link=*head
5. *head=temp



Algorithm for insertion at beg. 2

Insertion_Beg (node, info, next, start, ele)

1. new = (struct node *) malloc (size of (struct node))
2. new-> info = element
3. If start = NULL

   start = new

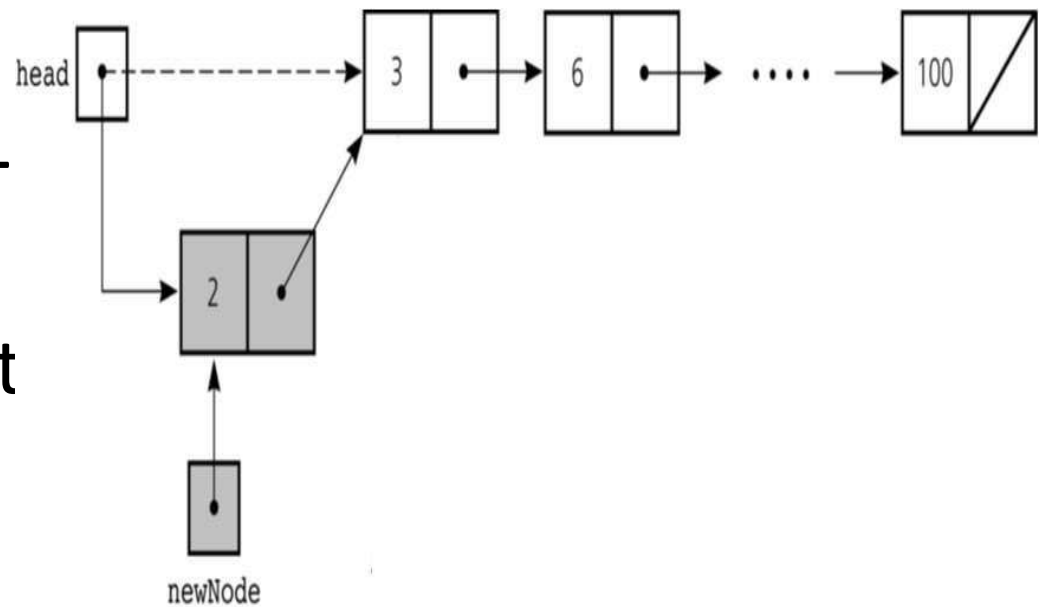
   new-> next = NUL

4. Else

   new-> next =start

   start = new

5. EXIT



Algorithm for insertion at end 1

- Insertion_End (temp, data , add, head, ptr)
- temp=(struct node *) malloc(sizeof(struct node))
- temp->data=data
- temp->link=NULL
- if(*head==NULL)
  *head=temp;

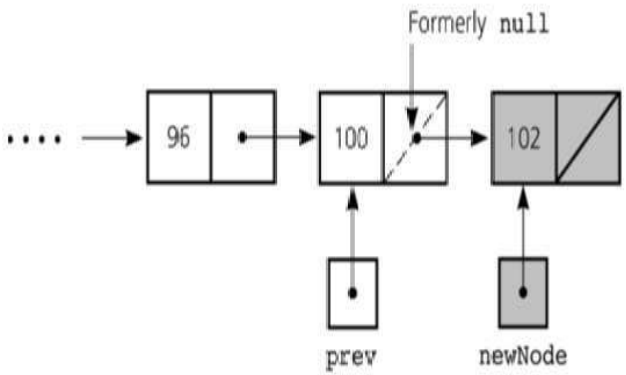
else

ptr=*head

while(ptr->link!=NULL)

ptr=ptr->link

- ptr->link=temp
- Exit



Algorithm for insertion at end 2

- Insertion_End (start, new, info, next, ptr, ele)

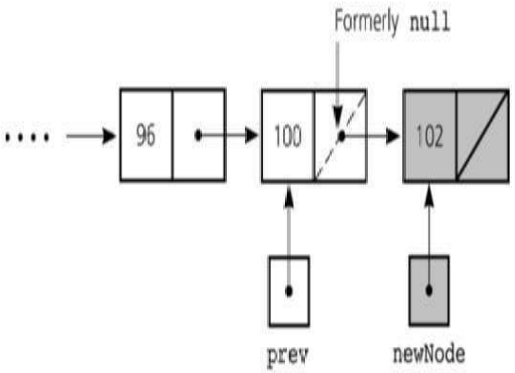
1. new= (struct node *) malloc (size of (struct node))
2. new-> info = ele
3. Initialize a pointer variable (ptr) from start

   ptr = start

   while (ptr-> next != NULL)

   ptr = ptr-> next // ptr =ptr++
4. new->next = ptr->next //NULL

   ptr-> next=new
5. Exit



Flip Video LINKS – www.imsuc.ac.in

- https://www.imsuc.ac.in/flp/schoolit_vid.php?subject=DATA%20STRUCTURE#

- https://www.imsuc.ac.in/flp/schoolit_vid.php?subject=DATA%20STRUCTURE#

- // for AVAIL

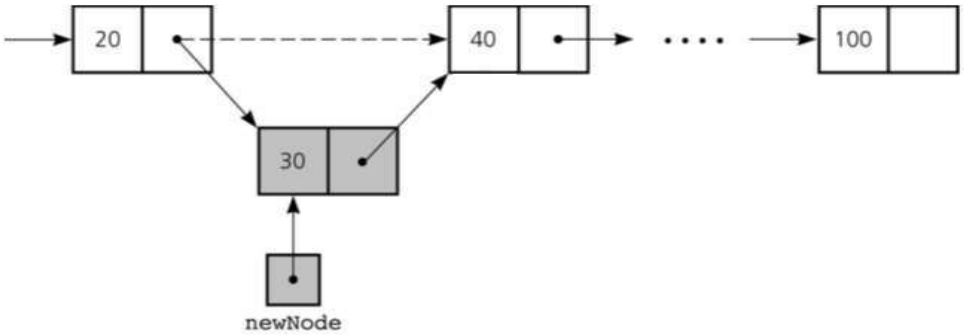
Algorithm for insertion at any position/ insertion after a given node

Insertion_pos (new, info, next, start, loc, pos, ele, i)

1. new= (struct node*)malloc(size of (struct node))
2. new-> info=ele
3. If start = NULL

   start=new

   new->next =NULL



4. Else

Initialize a pointer variable (loc) & counter variable (i) & set loc = start & i =0)

While (i < pos)

loc =loc –> next // loc = loc +1

i=i +1

5. new-> next=loc-> next

Loc->next=new

6. Exit

UNIT – III (LINKED LIST)
Deletion in linked list

Deleting a node in Single Linked List

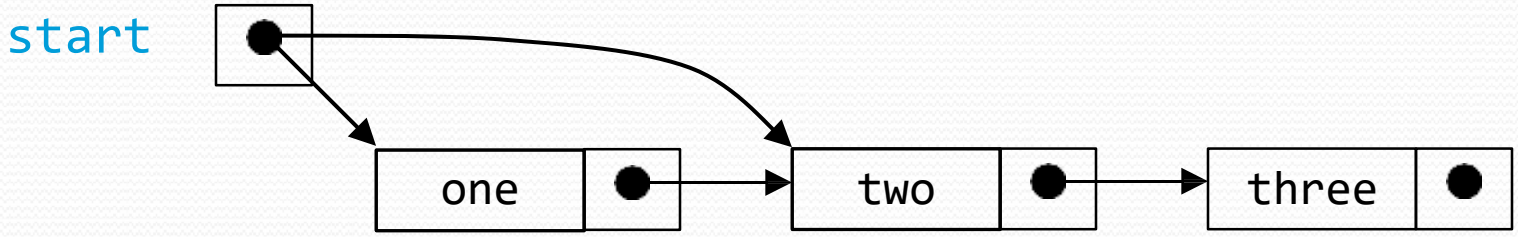
Here also we have threecases:-

- Deleting the first node
- Deleting the last node
- Deleting the intermediate node

Deleting the first node

Here we apply 2 steps:-

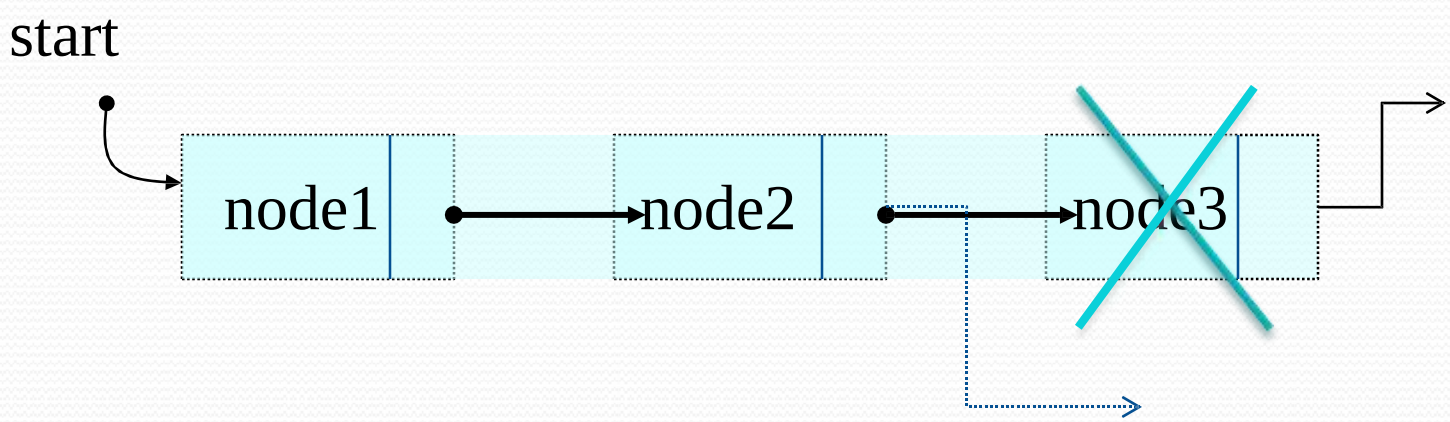
- Making the start pointer point towards the 2nd node
- Deleting the first node using delete keyword



Deleting the last node

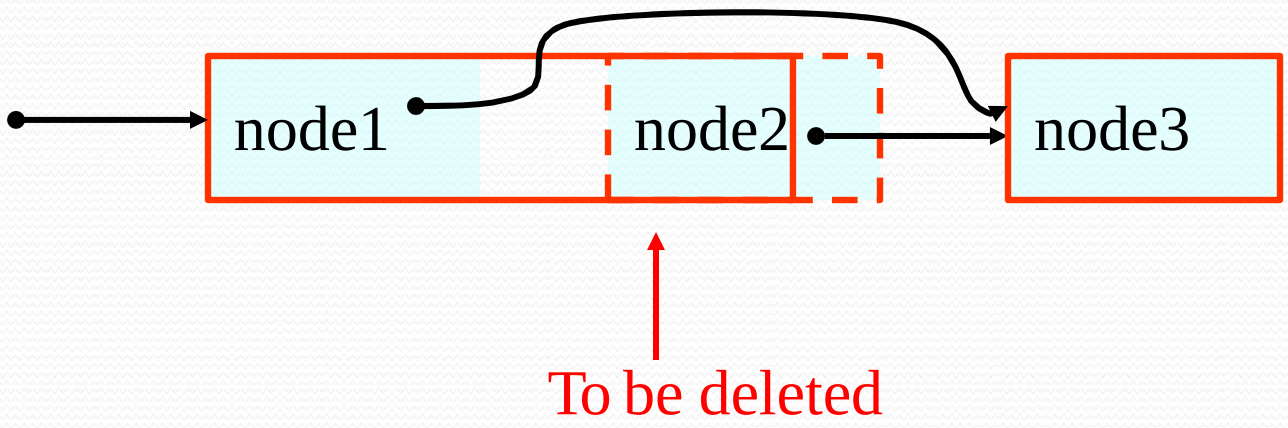
Here we apply 2 steps:-

- Making the second last node's next pointerpoint to NULL
- Deleting the last node viadelete keyword



Deleting a particular node

- Here we make the next pointer of the node previous to the node being deleted ,point to the successor node of the node to be deleted and then delete the node using delete keyword



Algorithm for deletion at beginning 1

- Del_beg (head, info, link, ptr, AVAIL)

1. If head = NULL then

   print Underflow & exit

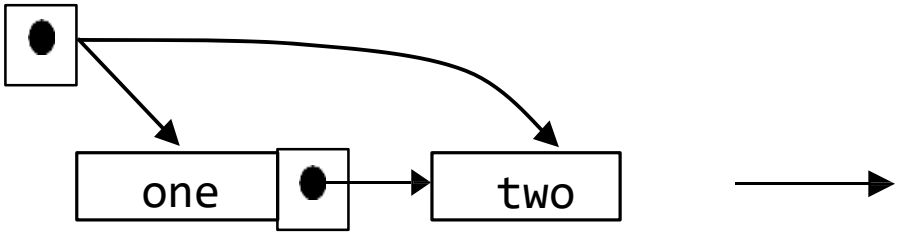
2. Else

   ptr = *head // now 1st node is ptr to be removed

   *head= ptr->link //now head is pointing to 2node

3. AVAIL =ptr OR free ptr

4. Exit



Algorithm for deletion at beginning 2

- Del_beg (start, info, next, loc, AVAIL)

1. If start = NULL then

   print Underflow & exit

2. Else

   loc = start // now 1st node is loc to be removed

   start = loc->next //now start is pointing to 2node

3. AVAIL =loc OR free loc

4. Exit

Algorithm for deletion at end 1

- Del_End (head, info, link, AVAIL, ptr, temp)

1. ptr=*head
2. if(ptr->link==NULL)

   head=NULL & free ptr

3. Else

   while(ptr->link!=NULL)

temp=ptr //

ptr=ptr->link // ptr = ptr ++

ptr=temp->link

4. temp->link= ptr-> link OR NULL
5. AVAIL = ptr or free ptr

Algorithm for deletion at end 2

- Del_End (start,info,next,AVAIL,loc,loc(p))

1. If start = NULL Then print underflow & exit
2. Else (take pointer variable loc & loc (p) & set loc = start)

   while (loc -> next != NULL)

   loc (p) = loc

   loc = loc (p) -> next // loc ++

3. Loc (p) -> next = loc -> next // NULL

   AVAIL = loc or free loc

4. Exit

Algorithm for deletion at any position 1

Del_pos (pos, ptr, temp, AVAIL, i)

1. If head = NULL then

   print overflow & exit

2. Else (initialize pointer variable ptr & temp & a counter variable i & set ptr=head , i=0)

3. While (i < pos)

temp=ptr

ptr= ptr->next // or ptr ++

i=i+1

4. temp->next = ptr->next

5. free ptr or AVAIL = ptr

6. Exit

Algorithm for deletion at any position 2

Del_loc (start, AVAIL, loc, loc(p), info, next)

1. If loc = start then
   start=loc->next // First node has deleted

2. Else
(set loc, loc(p) & a counter variable i)
Initialize loc=start, loc(p)=start, i=0

3. While (i < loc)
Loc(p)=loc
loc= loc(p)->next
i=i+1

4. loc(p)->next = loc->next // Creating a new link
   free loc OR AVAIL = loc
5. Exit

Algorithm for deletion when any specific element has given

- Write by your self (make some changes in deletion @ any position's algo.)

UNIT – III (LINKED LIST)
Traversing, Searching, Linked Stack

Traversing in linked list

- Traversing means accessing or visiting each node exactly once, in order to process it.
- Following algo. Traverse a singly linked list, started by START pointer & applies an operation (PROCESS) for traversing each node.
- The variable PTR holds the address of the node, currently being processed.

```
Traverse ( START, inf, next, ptr)
Step 1. Set ptr = START
Step2. while(ptr-> next != NULL)
        apply PROCESS to info. Part of ptr
        PROCESS = ptr->info.
        ptr=ptr++  // ptr= ptr->next
Step3  PROCESS = ptr->info.
Step4. Exit
```

Traversing in Doubly linked list

- We can traverse a D.L.L in 2 ways-

i) In forward direction ii) In backward direction

- Following algo. Traverse D.L.L. in forward direction start from first node to last, apply an operation (PROCESS) to each node with pointer variable ptr, where ptr holds the address of currently traversing node.

- Traverse_ DLL_forward(START, inf, next, ptr)

Step1. ptr=START

Step2. while (ptr-> next!= NULL)

apply PROCESS to inf. Part of ptr

PROCESS= ptr->inf.

ptr=ptr++ // ptr= ptr->next

Step3 PROCESS = ptr->info.

Step4. Exit

- Traverse_ DLL_backwrd(START,LAST,inf,prev,ptr)

Step1. ptr=LAST

Step2. while (ptr-> prev!= NULL)

apply PROCESS to inf. Part of ptr

PROCESS= ptr->inf.

ptr=ptr-- // ptr= ptr->prev

Step3 PROCESS = ptr->info.

Step4. Exit

Searching in linked list

- Searching means finding the location (loc) of an item in a linked list.
- It can be performed in following 2 ways-

i) Searching in unsorted linked list

ii) Searching in sorted linked list

- Let an (ITEM OR ELE) is to be searched. Following algo finds the loc of the node, where ITEM is first appears in singly linked list, which is un sorted.
- If the ITEM is not there in the list, then algo. Will set loc = NULL i.e search results in a unsuccessful search.

- Search(START,inf,ptr,ITEM,loc) // in unsorted list

```
Step1. set PTR=START
Step2. while (ptr-> next != NULL)
Step3. if (ptr->inf != ITEM)
            ptr=ptr ++  // ptr=ptr->next
       else if ptr->inf == ITEM
            set loc = ptr & exit // SUCCESSFULL Search
       else
        loc = NULL  & exit // UNSUCCESSFULL Search
```

- Search (START, inf,ptr,ITEM,loc) // in sorted linked list

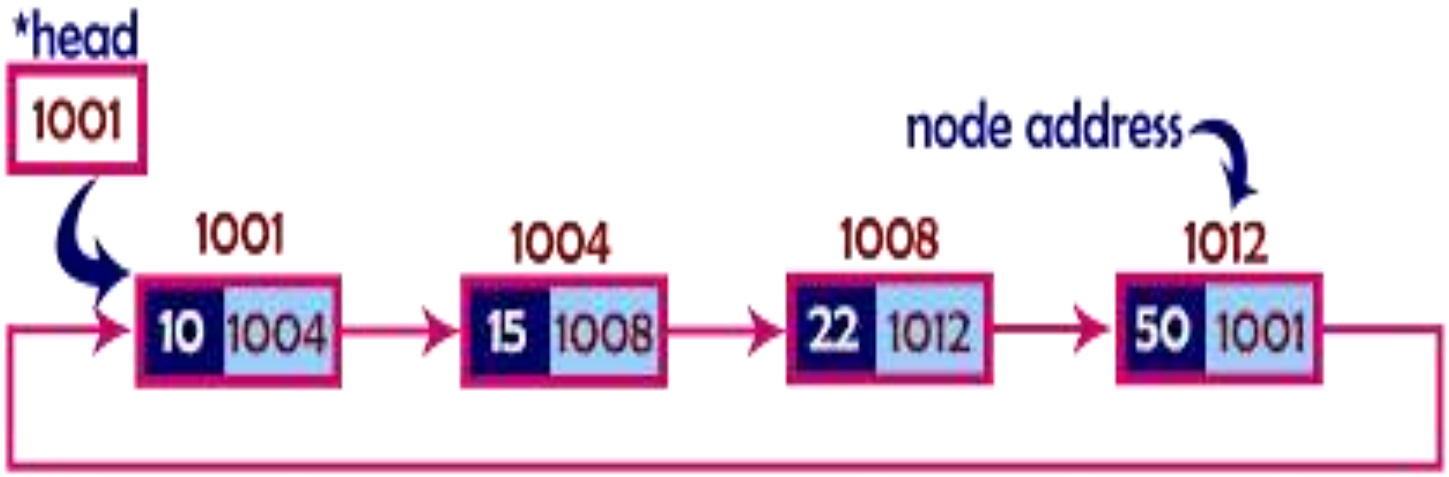
Step1. set ptr = START

Step2. while (ptr-> next != NULL)

Step3.

```
if      ITEM < ptr-> inf then //item is less then ptr->inf
        ptr= ptr++
else if
        ITEM== ptr-> inf then
        loc = ptr & exit   // SUCCESSFULL search
else
        loc = NULL & exit  // UNSUCCESSFUL search
```

Circular linked list



Circular linked list

- In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list.
- We can have circular singly linked list as well as circular doubly linked list.
- We traverse a circular singly linked list until we reach the same node where we started.
- The circular singly liked list has no beginning and no ending.
- There is no null value present in the next part of any of the nodes.

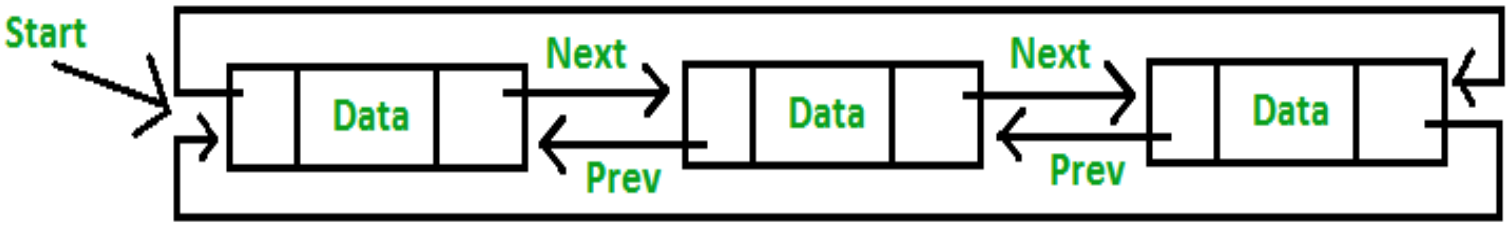
Basic Operations

- Following are the important operations supported by a circular list.

In this linked list insertion & deletion operations can be performed at bot the ends (start & end) but not in the between of linked list.

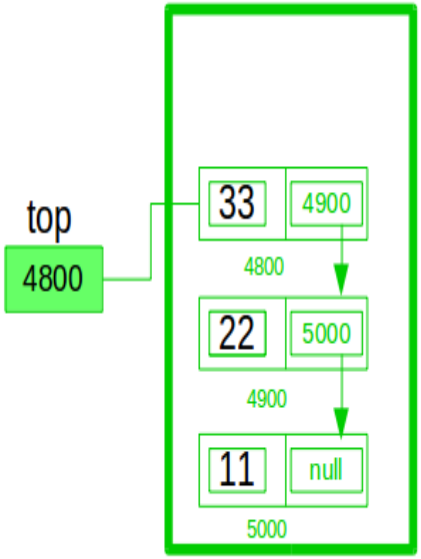
- **insert**
- **delete**
- **display** or **Traverse**

Recognize the Image ?

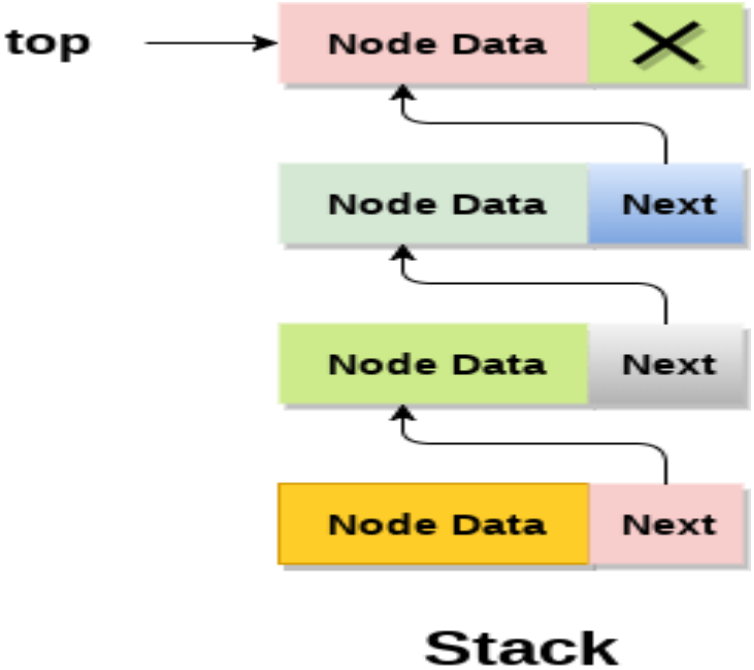


Linked Stack

- Stack can be represented in memory by f/w two ways- i) ii)
- When a stack is represented with the help of linked list, it is known as linked stack.
- The linear array based representation has various limitations – …..
- Whereas linked list based representation has various advantages as - …..



Initial Stack Having Three element
And top have address 4800



Linked Stack

- In this representation information part of node holds the element of stack & the link part of node contains the address of next node in linked list.

- The START pointer of linked list now behave as the TOS / TOP pointer variable of stack & NULL pointer of last node of linked list is at bottom of the stack.

Algo. For PUSH in linked stack

- Push_ls (TOP, info, next, ele, AVAIL)

Step 1. check for overflow

if AVAIL = NULL

write overflow & exit

Step 2. Remove a node from AVAIL using malloc

new = AVAIL & AVAIL = link[AVAIL]

Step 3. new-> info = ele

TOP = *new

Step 4. Exit

Algo. For POP in linked stack

- Pop_ls (TOP, ptr, info, next, ele, AVAIL)

Step 1. check for underflow

if TOP = NULL

write underflow & exit

Step 2. ptr= TOP

TOP = ptr->next

Step 3. AVAIL=ptr or free ptr

Step 4. Exit

UNIT – III (LINKED LIST)
Polynomial Representation

Representation of polynomial by using linked list

- Representing polynomial is one of the application of linked list.
- A polynomial is composed of different terms where each of them holds a coefficient and an exponent.
- A polynomial p(x) is the expression in variable x which is in the form ($ax^n + bx^{n-1} + ... + jx + k$), where a, b, c ...., k fall in the category of real numbers and 'n' is non negative integer, which is called the degree of polynomial.
- <u>An essential characteristic of the polynomial is that each term in the polynomial expression consists of two parts:</u>

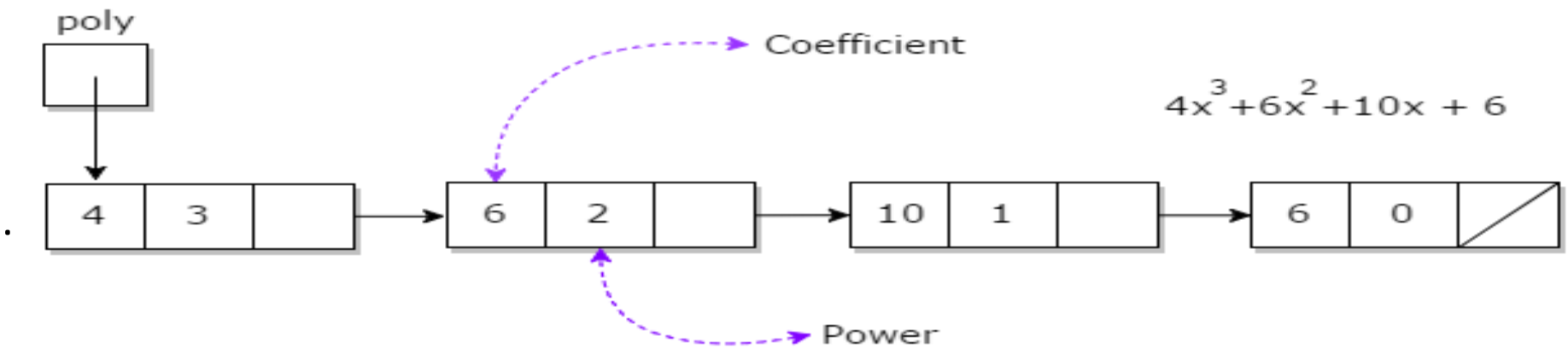
- one is the coefficient
- other is the exponent
- $10x^2 + 26x$, here 10 and 26 are coefficients and 2, 1 is its exponential value.

- <u>Points to keep in Mind while working with Polynomials:</u>

1. The sign of each coefficient and exponent is stored within the coefficient and the exponent itself

2. Additional terms having equal exponent is possible one

3. The storage allocation for each term in the polynomial must be done in ascending and descending order of their exponent



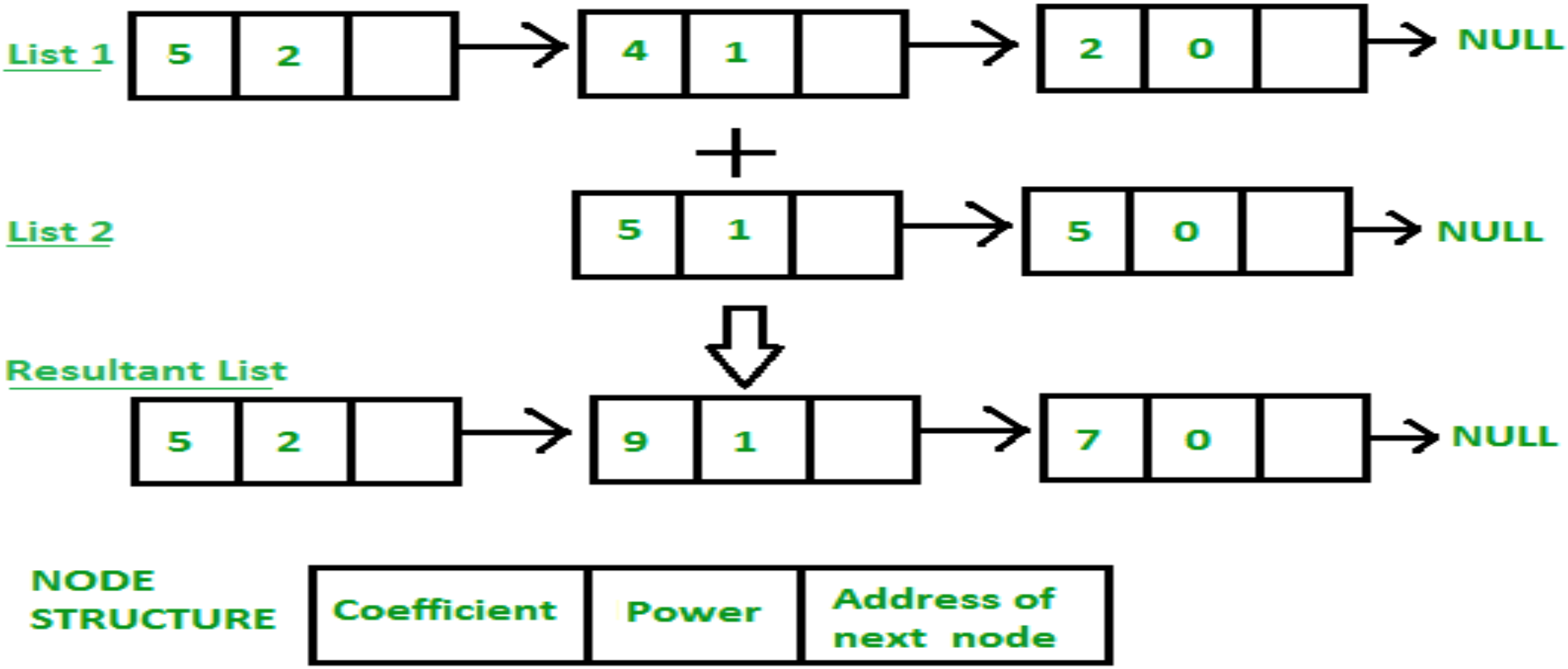
- <u>Polynomial can be represented in the various ways. These are:</u>
- By the use of arrays
- By the use of Linked List

- A polynomial can be thought of as an ordered list of non zero terms. Each non zero term is a two-tuple which holds two pieces of information:
- The exponent part
- The coefficient part

Perform Addition of given equations by using polynomial representation

5x2 + 4x + 2 with 5x + 5

Adding two polynomials using Linked List



Structure of polynomial node in C lang.

- struct polynomial node

```
{
int coffi;
Int expo;
Struct polynomial node *ptr
};
```

Advantages of Polynomials

- Expression / equations of any degree can be represented.
- Memory allocated & deallocated to the nodes only when it is required.
- We can use a header linked list to represent polynomials in memory.
- In linked list representation of polynomial nodes are linked in descending order only.

UNIT – III (LINKED LIST)
Garbage Collection

Garbage Collection (Avail)

- In computer science, garbage collection is a form of memory management, which is the responsibility of O.S.
- It automatically cleans up unused objects and pointers in memory, allowing the resources to be used again.
- The O.S time to time collect all the deleted & unused memory space in a free storage list & this technique is called as Garbage Collection.

- Some programming languages have built-in garbage collection, while others require custom functions to manage unused memory.
- Garbage collection may also be done at compile-time, when a program's source code is compiled into an executable program.
- Garbage collection is an important part of software development since it keeps programs from using up too much RAM.
- Besides helping programs run more efficiently, it can also prevent serious bugs, such as memory leaks, that can cause a program to crash.

- The garbage collection may take place when there is only some minimum amount of memory is available or when CPU is ideal & has time to do the collection.
- Maintaining the list in memory also require insertion of new nodes & hence require some technique which provide unused memory space for new node.
- As same as this technique is also require when any node is deleted so the lefted memory space become available for future.
- Together with the linked list in memory a special list is maintained which consists of un-used memory space (GARBAGE COLLECTION). This list has its own pointer i.e called as list of available space OR simply AVAIL.
- By using this AVAIL pointer the un-used memory cells will also be linked together to form a linked list.

Garbage Collection

- So, we can say that-
- If START = NULL // Denotes underflow in linked list
- If AVAIL = NULL // Denotes overflow in linked list

Algorithm for insertion at big. (with garbage collection)

- Ins_first (START, info, link, ele, AVAIL)
- Step 1 (check for OVERFLOW)

  if AVAIL= NULL

  print overflow & exit
- Step 2 (Remove a node from AVAIL list by using malloc function)

  new= AVAIL & AVAIL = link [AVAIL]
- Step 3 info [new] = ele

  link[new] = START

  START = new
- Step 4 Exit.